

IHK 2D-Array Aufgaben – Klassischer & Java-ähnlicher Pseudocode

Für jede Aufgabe siehst du zuerst den klassischen IHK-Pseudocode und direkt darunter eine Java-ähnliche Variante derselben Lösung.

Winter 2018/19 – Übungsaufgabe im Stil einer Raumbelegung

Aufgabe 1 – Raumbelegung prüfen

Für ein Schulungszentrum soll geprüft werden, ob ein bestimmter Raum an einem Tag in einem bestimmten Zeitslot frei ist.

Zu jedem Raum liegt ein Terminplan als zweidimensionales Integer-Array vor.

Jede Zeile enthält: Spalte 0 = Monat, Spalte 1 = Tag, Spalte 2–9 = Belegung der Zeitslots 0–7

(1 = belegt, 0 = frei).

Die Funktion `getRoomTable(int raumId)` liefert für eine Raum-ID den Terminplan.

Schreiben Sie eine Funktion `boolean getRoomFree(int month, int day, int raumId, int slot)`,

die `true` zurückgibt, wenn der Raum an diesem Tag im gewünschten Slot frei ist.

Wenn kein bestimmter Slot ausgewählt werden muss, hat der Parameter `slot` den Wert `-1`

und es genügt, irgendeinen freien Slot zu finden.

Klassischer Pseudocode:

```
funktion getRoomFree(month: Integer, day: Integer,
                    raumId: Integer, slot: Integer) Boolean

    plan ← getRoomTable(raumId)

    FOR i ← 0 TO LENGTH(plan) - 1 DO
        // passenden Tag suchen
        IF plan[i][0] = month AND plan[i][1] = day THEN

            IF slot = -1 THEN                                // irgendein Slot reicht
                FOR s ← 0 TO 7 DO
                    IF plan[i][2 + s] = 0 THEN
                        RETURN true
```

```

        ENDIF
    ENDFOR
ELSE // bestimmter Slot
    IF plan[i][2 + slot] = 0 THEN
        RETURN true
    ENDIF
ENDIF

ENDIF
ENDFOR

RETURN false
endfunktion

```

Java-ähnlicher Pseudocode:

```

boolean getRoomFree(int month, int day, int raumId, int slot) {

    int[][] plan = getRoomTable(raumId);

    for (int i = 0; i < plan.length; i++) {
        // passenden Tag suchen
        if (plan[i][0] == month && plan[i][1] == day) {

            if (slot == -1) { // irgendein Slot reicht
                for (int s = 0; s <= 7; s++) {
                    if (plan[i][2 + s] == 0) {
                        return true;
                    }
                }
            } else { // bestimmter Slot
                if (plan[i][2 + slot] == 0) {
                    return true;
                }
            }
        }
    }

    return false;
}

```

Aufgabe 2 – Schicht eines Mitarbeiters prüfen

In einem Unternehmen wird ein Schichtplan geführt. Für jeden Mitarbeiter gibt es ein zweidimensionales Array mit Schichtinformationen.

Jede Zeile: Spalte 0 = Tag, Spalte 1–4 = Schichten 0–3 (1 = Mitarbeiter arbeitet, 0 = frei).

Die Funktion `getShiftTable(int id)` liefert den Plan eines Mitarbeiters.

Entwickeln Sie eine Funktion `boolean isShiftFree(int day, int id, int schicht)`, die zurückgibt, ob der Mitarbeiter an diesem Tag in der angegebenen Schicht frei ist.

Klassischer Pseudocode:

```
funktion isShiftFree(day: Integer, id: Integer, schicht: Integer) Boolean

    plan ← getShiftTable(id)

    FOR i ← 0 TO LENGTH(plan) - 1 DO
        IF plan[i][0] = day THEN
            IF plan[i][1 + schicht] = 0 THEN
                RETURN true
            ELSE
                RETURN false
            ENDIF
        ENDIF
    ENDFOR

    // Tag nicht im Plan gefunden
    RETURN false
endfunktion
```

Java-ähnlicher Pseudocode:

```
boolean isShiftFree(int day, int id, int schicht) {

    int[][] plan = getShiftTable(id);

    for (int i = 0; i < plan.length; i++) {
        if (plan[i][0] == day) {
            if (plan[i][1 + schicht] == 0) {
                return true;
            } else {
                return false;
            }
        }
    }
}
```

```
        }  
    }  
  
    return false;  
}
```

Aufgabe 3 – Funktion getAppointment

Für eine Gemeinschaftspraxis mit mehreren Ärzten soll geprüft werden, ob für einen Arzt an einem

bestimmten Tag ein Termin gefunden werden kann.

Für jeden Arzt liegt ein zweidimensionales Array vor: Spalte 0 = Monat, Spalte 1 = Tag, Spalte 2–9 = Zeitslots 0–7 (1 = belegt, 0 = frei).

Die Funktion getTimeTable(int id) liefert den Plan eines Arztes.

Schreiben Sie eine Funktion boolean getAppointment(int month, int day, int id, int slot),

die true liefert, wenn für den gegebenen Tag ein freier Termin gefunden werden kann.

Hat der Parameter slot den Wert -1, muss kein bestimmter Zeitslot ausgewählt werden.

Klassischer Pseudocode:

```
funktion getAppointment(month: Integer, day: Integer,
                        id: Integer, slot: Integer) Boolean

    tabelle ← getTimeTable(id)

    FOR i ← 0 TO LENGTH(tabelle) - 1 DO
        IF tabelle[i][0] = month AND tabelle[i][1] = day THEN

            IF slot = -1 THEN                // irgendein Slot für diesen Tag
                FOR s ← 0 TO 7 DO
                    IF tabelle[i][2 + s] = 0 THEN
                        RETURN true
                    ENDIF
                ENDFOR
            ELSE                            // bestimmter Slot
                IF tabelle[i][2 + slot] = 0 THEN
                    RETURN true
                ENDIF
            ENDIF
        ENDIF
    ENDFOR

    RETURN false
endfunktion
```

Java-ähnlicher Pseudocode:

```
boolean getAppointment(int month, int day, int id, int slot) {

    int[][] table = getTimeTable(id);

    for (int i = 0; i < table.length; i++) {
        if (table[i][0] == month && table[i][1] == day) {

            if (slot == -1) {                // irgendein Slot für diesen Tag
                for (int s = 0; s <= 7; s++) {
                    if (table[i][2 + s] == 0) {
                        return true;
                    }
                }
            } else {                        // bestimmter Slot
                if (table[i][2 + slot] == 0) {
                    return true;
                }
            }
        }
    }

    return false;
}
```

Aufgabe 4 – Parkplatz-Zeitslot prüfen

In einem Parkhaus sollen Reservierungen für Parkplätze verwaltet werden. Für jedes Parkhaus

gibt es ein zweidimensionales Array. Jede Zeile steht für einen Tag; Spalte 0 = Tag, Spalte 1–10 = Parkplätze 1–10 (1 = belegt, 0 = frei).

Die Funktion `getParkingTable(int hausId)` liefert den Plan.

Entwickeln Sie eine Funktion `boolean getFreeParking(int day, int hausId, int platz)`, die prüft, ob ein bestimmter Parkplatz an diesem Tag frei ist. Ist `platz == -1`, soll ein beliebiger freier Parkplatz genügen.

Klassischer Pseudocode:

```
funktion getFreeParking(day: Integer, hausId: Integer, platz: Integer) Boolean

    plan ← getParkingTable(hausId)

    FOR i ← 0 TO LENGTH(plan) - 1 DO
        IF plan[i][0] = day THEN

            IF platz = -1 THEN
                FOR p ← 1 TO 10 DO
                    IF plan[i][p] = 0 THEN
                        RETURN true
                    ENDIF
                ENDFOR
            ELSE
                IF plan[i][platz] = 0 THEN
                    RETURN true
                ENDIF
            ENDIF
        ENDIF
    ENDFOR

    RETURN false
endfunktion
```

Java-ähnlicher Pseudocode:

```
boolean getFreeParking(int day, int hausId, int platz) {
```

```
int[][] plan = getParkingTable(hausId);

for (int i = 0; i < plan.length; i++) {
    if (plan[i][0] == day) {

        if (platz == -1) {
            for (int p = 1; p <= 10; p++) {
                if (plan[i][p] == 0) {
                    return true;
                }
            }
        } else {
            if (plan[i][platz] == 0) {
                return true;
            }
        }
    }
}

return false;
}
```


Aufgabe 5 – Schulungsraum-Reservierung prüfen

Ein Bildungsträger verwaltet Schulungsräume. Für jeden Raum liegt ein Terminplan vor, der pro Zeile Monat, Tag und die Belegung der Slots 0–5 enthält (Spalte 0 = Monat, Spalte 1 = Tag, Spalte 2–7 = Slots 0–5).

Die Funktion `getRoomSchedule(int raumId)` liefert diesen Plan.

Schreiben Sie eine Funktion `boolean hasFreeTrainingSlot(int month, int day, int raumId, int slot)`, die prüft, ob ein Schulungsraum an einem Tag frei ist. Ist `slot == -1`, genügt ein beliebiger freier Slot.

Klassischer Pseudocode:

```
funktion hasFreeTrainingSlot(month: Integer, day: Integer,  
                             raumId: Integer, slot: Integer) Boolean  
  
    plan ← getRoomSchedule(raumId)  
  
    FOR i ← 0 TO LENGTH(plan) - 1 DO  
        IF plan[i][0] = month AND plan[i][1] = day THEN  
  
            IF slot = -1 THEN  
                FOR s ← 0 TO 5 DO  
                    IF plan[i][2 + s] = 0 THEN  
                        RETURN true  
                    ENDIF  
                ENDFOR  
            ELSE  
                IF plan[i][2 + slot] = 0 THEN  
                    RETURN true  
                ENDIF  
            ENDIF  
        ENDIF  
    ENDFOR  
  
    RETURN false  
endfunktion
```

Java-ähnlicher Pseudocode:

```
boolean hasFreeTrainingSlot(int month, int day, int raumId, int slot) {

    int[][] plan = getRoomSchedule(raumId);

    for (int i = 0; i < plan.length; i++) {
        if (plan[i][0] == month && plan[i][1] == day) {

            if (slot == -1) {
                for (int s = 0; s <= 5; s++) {
                    if (plan[i][2 + s] == 0) {
                        return true;
                    }
                }
            } else {
                if (plan[i][2 + slot] == 0) {
                    return true;
                }
            }
        }
    }

    return false;
}
```

Aufgabe 6 – Freier Arbeitsplatz im Computerlabor

In einem Computerlabor gibt es mehrere Arbeitsplätze. Für jeden Tag wird ein zweidimensionales

Array geführt: Zeile = Arbeitsplatznummer, Spalte 0 = Arbeitsplatz-ID, Spalte 1–8 = Zeitslots (1 = belegt, 0 = frei).

Die Funktion `getLabTable(int labId)` liefert den Plan für ein Labor.

Entwickeln Sie eine Funktion `boolean isWorkstationFree(int labId, int platzId, int slot)`, die prüft, ob der angegebene Platz in einem bestimmten Slot frei ist.

Klassischer Pseudocode:

```
funktion isWorkstationFree(labId: Integer, platzId: Integer, slot: Integer) Boolean

    plan ← getLabTable(labId)

    FOR i ← 0 TO LENGTH(plan) - 1 DO
        IF plan[i][0] = platzId THEN
            IF plan[i][1 + slot] = 0 THEN
                RETURN true
            ELSE
                RETURN false
            ENDIF
        ENDIF
    ENDFOR

    RETURN false
endfunktion
```



Java-ähnlicher Pseudocode:

```
boolean isWorkstationFree(int labId, int platzId, int slot) {

    int[][] plan = getLabTable(labId);

    for (int i = 0; i < plan.length; i++) {
        if (plan[i][0] == platzId) {
            if (plan[i][1 + slot] == 0) {
                return true;
            } else {
                return false;
            }
        }
    }

    return false;
}
```

```
        }  
    }  
}  
  
return false;  
}
```

Aufgabe 7 – Freier Kinositz

In einem Kino sollen Sitzplätze für eine Vorstellung verwaltet werden. Für jede Vorstellung

gibt es ein zweidimensionales Array, in dem jede Zeile einen Sitz beschreibt:

Spalte 0 = Reihen-Nummer, Spalte 1 = Sitz-Nummer, Spalte 2 = Belegung (1 = belegt, 0 = frei).

Die Funktion `getShowPlan(int showId)` liefert diesen Plan.

Schreiben Sie eine Funktion `boolean isSeatFree(int showId, int reihe, int sitz)`, die zurückgibt, ob ein bestimmter Sitz in einer Vorstellung frei ist.

Klassischer Pseudocode:

```
funktion isSeatFree(showId: Integer, reihe: Integer, sitz: Integer) Boolean

    plan ← getShowPlan(showId)

    FOR i ← 0 TO LENGTH(plan) - 1 DO
        IF plan[i][0] = reihe AND plan[i][1] = sitz THEN
            IF plan[i][2] = 0 THEN
                RETURN true
            ELSE
                RETURN false
            ENDIF
        ENDIF
    ENDFOR

    RETURN false
endfunktion
```

Java-ähnlicher Pseudocode:

```
boolean isSeatFree(int showId, int reihe, int sitz) {

    int[][] plan = getShowPlan(showId);

    for (int i = 0; i < plan.length; i++) {
        if (plan[i][0] == reihe && plan[i][1] == sitz) {
            if (plan[i][2] == 0) {
                return true;
            } else {
                return false;
            }
        }
    }

    return false;
}
```

```
        }  
    }  
}  
  
return false;  
}
```